

2.01. Le rond

2.01 : le rond — un cercle. Un dessin de 16×16 posé au milieu de l'écran avec `sprite16()`, son nom dessous.

1. Ce qu'il faut comprendre

Un rond (on dit aussi un cercle), c'est une forme sans côté et sans coin. Tous les points du bord sont à **la même distance du centre** : cette distance s'appelle le **rayon**. Deux rayons bout à bout, d'un bord à l'autre en passant par le centre, font le **diamètre**.

Sur une grille, un rond n'est jamais tout à fait rond : les pixels sont des petits carrés. On l'approche en marches d'escalier. Regarde les rangées : en haut, 6 pixels de large ; puis 10, 12, 14 ; au milieu, les 16 pixels. Les marches sont plus courtes vers le haut et le bas, plus longues sur les côtés : c'est ce qui donne l'impression d'une courbe.

Cette série dessine des formes géométriques, une par leçon. Le programme reste le même d'une leçon à l'autre : seuls changent le dessin et le mot. Tu peux donc te concentrer sur la forme.

Le dessin est dans son propre fichier, `perso_ROND.cpp` (l'onglet à côté de `principal.cpp`). Il fait **16×16 pixels** : 16 rangées de 16 signes. `#` est le plus sombre (le contour), `-` est clair (l'intérieur), `.` est le **transparent** : dans un personnage, on voit le fond de l'écran au travers.

Pourquoi 16×16 et pas 8×8 ? Dans une tuile de 8 pixels, un rond aurait 4 pixels de rayon : il ressemblerait à un carré aux coins cassés. Avec 16 pixels, la courbe se voit.

Les trois lignes `#include` : `<texte>` pour écrire le nom, `<sprite16>` pour poser un dessin de 16×16 , `<Perso>` pour avoir le droit d'écrire un `Perso`. Puis `#include "perso_ROND.cpp"` verse le dessin dans le programme (des guillemets, parce que c'est un fichier du projet, pas une fonction de la console).

`sprite16(0, 72, 56, ROND)` pose le dessin avec les lutins 0, 1, 2 et 3 (un personnage de 16×16 en prend quatre). 72 et 56 sont la place du **coin en haut à gauche**, en pixels. L'écran fait 160 pixels de large : $(160 - 16) / 2 = 72$, c'est le milieu.

`texte(8, 10, "ROND")` écrit le nom sous la forme. Ici on compte en **cases de 8 pixels** : la ligne 10 est à 80 pixels du haut, juste sous la forme (qui s'arrête à 72).

La boucle du jeu ne fait qu'attendre l'image suivante : rien ne bouge, on regarde.

Essaie : dans `perso_ROND.cpp`, remplace tous les `-` par des `+` : le rond devient plein et plus foncé.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <sprite16>` → [35.2. La fonction `sprite16\(\)` — un personnage de \$16 \times 16\$](#)
- `#include <Perso>` → [35.1. Perso — dessiner des personnages](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **488 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l’a demandé — et ce qu’il a laissé, parce que rien ne l’appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l’allumage	– la mise en route de la console
main()	117 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	– attend l’image suivante
AttendreVBlank	16 o	← démarrage	– attend que l’écran soit libre
AttendreAcces	12 o	← EcrireTexte	– attend de pouvoir écrire une case
EcrireTexte	10 o	← texte()	– écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage	– range les 40 lutins hors de l’écran
InstallerTransfert	15 o	← démarrage	– installe la copie des lutins en mémoire rapide
CopierTuiles	58 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l’écran
Tuiles	128 o	← CopierTuiles	– 4 lettres sur 43, et 4 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert	– la petite routine qui copie les lutins
"ROND"	4 o	← texte()	– un texte gravé

Lettres gravées : DNOR

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

sprite()	× 4	96 o	sur place, dans main()
texte()	× 1	11 o	sur place, dans main() + EcrireTexte (10 o)
image()	× 1	6 o	sur place, dans main() + AttendreImage (28 o)
sprite16()	× 1	0 o	sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- 2.01 Le rond ----
// Le rond de 16 × 16 au milieu de l'écran, et son nom dessous.

#include <texte>           // texte() : écrire un mot à l'écran
#include <sprite16>        // sprite16() : poser un personnage de 16 × 16
#include <Perso>           // Perso : un dessin de 16 × 16
#include "perso_ROND.cpp"  // le dessin, dans son fichier
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    sprite16(0, 72, 56, ROND);           // les lutins 0 à 3, coin haut-gauche en (72, 56)
    texte(8, 10, "ROND");               // le nom, sous la forme

    while (true) { // la boucle du jeu : rien ne bouge, on regarde
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n’est pas montrée — un programme qui ne bouge pas n’a qu’une seule image, et c’est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le rond au milieu de l'écran, et ROND écrit dessous.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la forme est posée au milieu, en (72, 56) (72, 56)
- ✓ ses quatre lutins font un carré de 16
- ✓ son nom est écrit dessous : ROND

Ensuite : **2.02. Le carré** — série 2 : Les formes géométriques. · [gameboy3](#) — écrire une cartouche Game Boy en C++